

```
In [1]: !pip install pandas matplotlib seaborn numpy IPython
```

Collecting pandas

Using cached pandas-3.0.3-cp314-cp314-macosx_11_0_arm64.whl.metadata (79 kB)

Collecting matplotlib

Using cached matplotlib-3.10.9-cp314-cp314-macosx_11_0_arm64.whl.metadata (52 kB)

Collecting seaborn

Using cached seaborn-0.13.2-py3-none-any.whl.metadata (5.4 kB)

Collecting numpy

Using cached numpy-2.4.6-cp314-cp314-macosx_14_0_arm64.whl.metadata (6.6 kB)

Requirement already satisfied: IPython in ./venv_html/lib/python3.14/site-packages (9.14.0)

Requirement already satisfied: python-dateutil>=2.8.2 in ./venv_html/lib/python3.14/site-packages (from pandas) (2.9.0.post0)

Collecting contourpy>=1.0.1 (from matplotlib)

Using cached contourpy-1.3.3-cp314-cp314-macosx_11_0_arm64.whl.metadata (5.5 kB)

Collecting cycler>=0.10 (from matplotlib)

Using cached cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)

Collecting fonttools>=4.22.0 (from matplotlib)

Using cached fonttools-4.63.0-cp314-cp314-macosx_10_15_universal2.whl.metadata (118 kB)

Collecting kiwisolver>=1.3.1 (from matplotlib)

Using cached kiwisolver-1.5.0-cp314-cp314-macosx_11_0_arm64.whl.metadata (5.1 kB)

Requirement already satisfied: packaging>=20.0 in ./venv_html/lib/python3.14/site-packages (from matplotlib) (26.2)

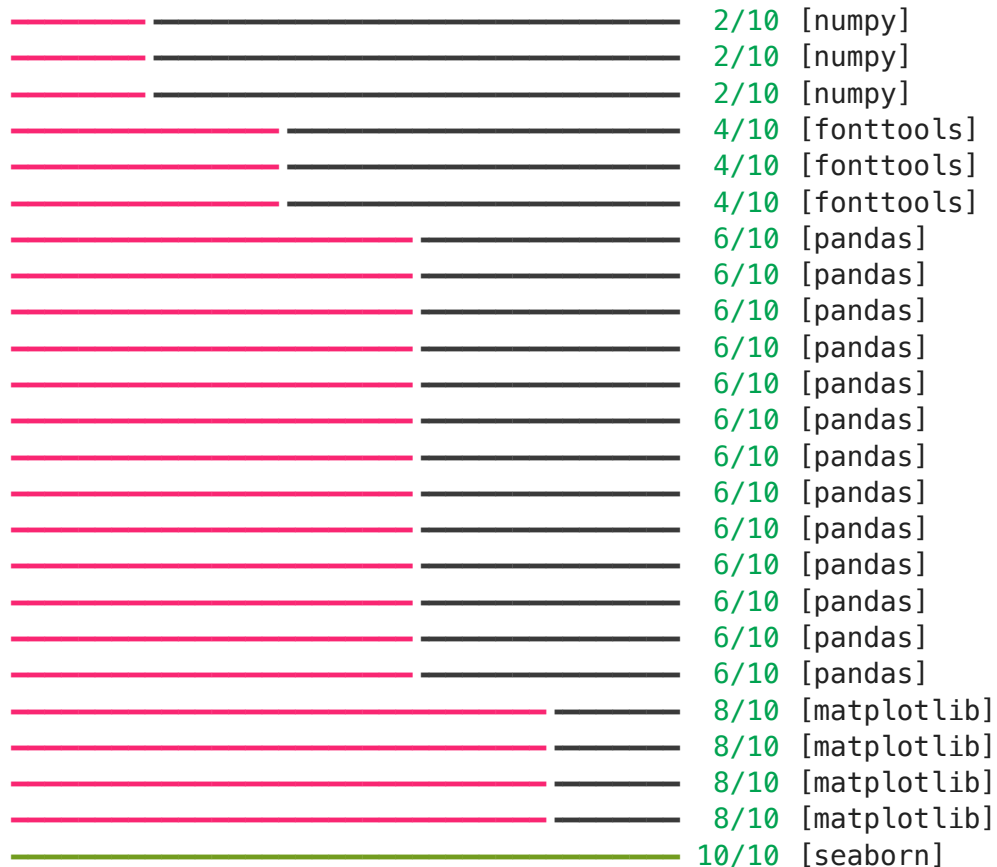
Collecting pillow>=8 (from matplotlib)

Using cached pillow-12.2.0-cp314-cp314-macosx_11_0_arm64.whl.metadata (8.8 kB)

```

Collecting pyparsing>=3 (from matplotlib)
  Using cached pyparsing-3.3.2-py3-none-any.whl.metadata (5.8 kB)
Requirement already satisfied: decorator>=5.1.0 in ./venv_html/lib/
python3.14/site-packages (from IPython) (5.3.1)
Requirement already satisfied: ipython-pygments-lexers>=1.0.0 in ./
venv_html/lib/python3.14/site-packages (from IPython) (1.1.1)
Requirement already satisfied: jedi>=0.18.2 in ./venv_html/lib/pytho
n3.14/site-packages (from IPython) (0.20.0)
Requirement already satisfied: matplotlib-inline>=0.1.6 in ./venv_h
tml/lib/python3.14/site-packages (from IPython) (0.2.2)
Requirement already satisfied: pexpect>4.6 in ./venv_html/lib/pytho
n3.14/site-packages (from IPython) (4.9.0)
Requirement already satisfied: prompt_toolkit<3.1.0,>=3.0.41 in ./v
env_html/lib/python3.14/site-packages (from IPython) (3.0.52)
Requirement already satisfied: psutil>=7 in ./venv_html/lib/python
3.14/site-packages (from IPython) (7.2.2)
Requirement already satisfied: pygments>=2.14.0 in ./venv_html/lib/
python3.14/site-packages (from IPython) (2.20.0)
Requirement already satisfied: stack_data>=0.6.0 in ./venv_html/li
b/python3.14/site-packages (from IPython) (0.6.3)
Requirement already satisfied: traitlets>=5.13.0 in ./venv_html/li
b/python3.14/site-packages (from IPython) (5.15.0)
Requirement already satisfied: wcwidth in ./venv_html/lib/python3.1
4/site-packages (from prompt_toolkit<3.1.0,>=3.0.41->IPython) (0.7.
0)
Requirement already satisfied: parso<0.9.0,>=0.8.6 in ./venv_html/l
ib/python3.14/site-packages (from jedi>=0.18.2->IPython) (0.8.7)
Requirement already satisfied: ptyprocess>=0.5 in ./venv_html/lib/p
ython3.14/site-packages (from pexpect>4.6->IPython) (0.7.0)
Requirement already satisfied: six>=1.5 in ./venv_html/lib/python3.
14/site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: executing>=1.2.0 in ./venv_html/lib/
python3.14/site-packages (from stack_data>=0.6.0->IPython) (2.2.1)
Requirement already satisfied: asttokens>=2.1.0 in ./venv_html/lib/
python3.14/site-packages (from stack_data>=0.6.0->IPython) (3.0.1)
Requirement already satisfied: pure-eval in ./venv_html/lib/python
3.14/site-packages (from stack_data>=0.6.0->IPython) (0.2.3)
Using cached pandas-3.0.3-cp314-cp314-macosx_11_0_arm64.whl (9.9 MB)
Using cached matplotlib-3.10.9-cp314-cp314-macosx_11_0_arm64.whl (8.
2 MB)
Using cached seaborn-0.13.2-py3-none-any.whl (294 kB)
Using cached numpy-2.4.6-cp314-cp314-macosx_14_0_arm64.whl (5.2 MB)
Using cached contourpy-1.3.3-cp314-cp314-macosx_11_0_arm64.whl (273
kB)
Using cached cycler-0.12.1-py3-none-any.whl (8.3 kB)
Using cached fonttools-4.63.0-cp314-cp314-macosx_10_15_universal2.wh
l (2.9 MB)
Using cached kiwisolver-1.5.0-cp314-cp314-macosx_11_0_arm64.whl (64
kB)
Using cached pillow-12.2.0-cp314-cp314-macosx_11_0_arm64.whl (4.7 M
B)
Using cached pyparsing-3.3.2-py3-none-any.whl (122 kB)
Installing collected packages: pyparsing, pillow, numpy, kiwisolver,
fonttools, cycler, pandas, contourpy, matplotlib, seaborn
  _____ 1/10 [pillow]
  _____ 2/10 [numpy]

```



Successfully installed contourpy-1.3.3 cycler-0.12.1 fonttools-4.63.0 kiwisolver-1.5.0 matplotlib-3.10.9 numpy-2.4.6 pandas-3.0.3 pillow-12.2.0 pyparsing-3.3.2 seaborn-0.13.2

[notice] A new release of pip is available: 25.3 -> 26.1.2

[notice] To update, run: pip install --upgrade pip

```
In [2]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import os
from IPython.display import display, Markdown

# Set style for all plots
plt.style.use('seaborn-v0_8-whitegrid')
sns.set_palette('viridis')
plt.rcParams['figure.figsize'] = (10, 6)

# Paths to CSVs
BASELINE_DIR = '../Results/baseline'
ESTIMATED_DIR = '../Results/estimated'
PLOT_DIR = 'plot'
os.makedirs(PLOT_DIR, exist_ok=True)

# Load data
df_base_aicore2 = pd.read_csv(f'{BASELINE_DIR}/performance_baseline_2.csv')
df_base_aicore4 = pd.read_csv(f'{BASELINE_DIR}/performance_baseline_4.csv')
df_base_cubemx = pd.read_csv(f'{BASELINE_DIR}/performance_baseline_cubemx.csv')
df_base_aistudio = pd.read_csv(f'{BASELINE_DIR}/performance_baseline_aistudio.csv')
```

```
df_est_aistudio = pd.read_csv(f'{ESTIMATED_DIR}/performance_estimates.csv')

# Pre-processing: convert Inference Time to numeric (coerces 'FAIL' to NaN)
for df in [df_base_aicore2, df_base_aicore4, df_base_cubemx, df_base_cubemx4]:
    df['Inference Time (ms)'] = pd.to_numeric(df['Inference Time (ms)'], errors='coerce')
```

NPU Performance Analysis

This notebook tells the story of the STM32N6 NPU capabilities, illustrating exactly what it accelerates and where its physical limits lie. We will explore the raw performance of the most modern tool, **AI STUDIO**, breaking down its optimizations and bottlenecks. Then, we will compare the compiler evolution (v2.2 vs v4.0) both locally and in the cloud. Finally, we will analyze the reliability of the compiler estimates versus actual hardware execution.

0. Overview of Tested Models and CPU Fallback

Before diving into performance metrics, let's look at the dataset. We tested multiple single-layer sub-models to stress-test specific features of the NPU (various kernel sizes, point-wise, depth-wise, and pooling layers).

```
In [3]: overview_df = df_base_aistudio[['Model Name', 'Layer Type', 'Filter Size', 'Kernel Size', 'Stride', 'Padding', 'Dilation', 'Grouping', 'Output Channels', 'CPU Fallback (%)']]
overview_df['Model Name'] = overview_df['Model Name'].str.replace(' ', '_')
overview_df.sort_values(by='MACC', inplace=True)
overview_df.reset_index(drop=True, inplace=True)

display(overview_df)

cpu_fallback_mean = pd.to_numeric(overview_df['CPU Fallback (%)'], errors='coerce').mean()
display(Markdown(f"**Key Takeaway:** As shown in the table above, the average CPU fallback is {cpu_fallback_mean}%"))
```

	Model Name	Layer Type	Filters	Kernel Size	MACC	CPU Fallback (%)	Allo
0	depthwise_3x3	DepthwiseConv2D	3	3x3	122883	0	
1	conv2d_1x1	Conv2D (Pointwise)	32	1x1	524320	0	
2	conv2d_16f_3x3	Conv2D	16	3x3	1835024	0	
3	conv2d_poll	Conv2D + MaxPool	16	3x3	1851408	0	
4	conv2d_3x3	Conv2D	32	3x3	3670048	0	
5	conv2d_64f_3x3	Conv2D	64	3x3	7340096	0	
6	conv2d_32f_5x5	Conv2D	32	5x5	9961504	0	
7	conv2d_32f_7x7	Conv2D	32	7x7	19398688	0	

Key Takeaway: As shown in the table above, the NPU Allocation is consistently 100% and there is **0.0% CPU Fallback**. This means the STM32N6 NPU natively supports and accelerates all these layer types (Conv2D, Depthwise, MaxPool) entirely in hardware. The Cortex-M CPU never has to intervene to compute unsupported operators.

Part 1: AI STUDIO - Unveiling the Modern Tool

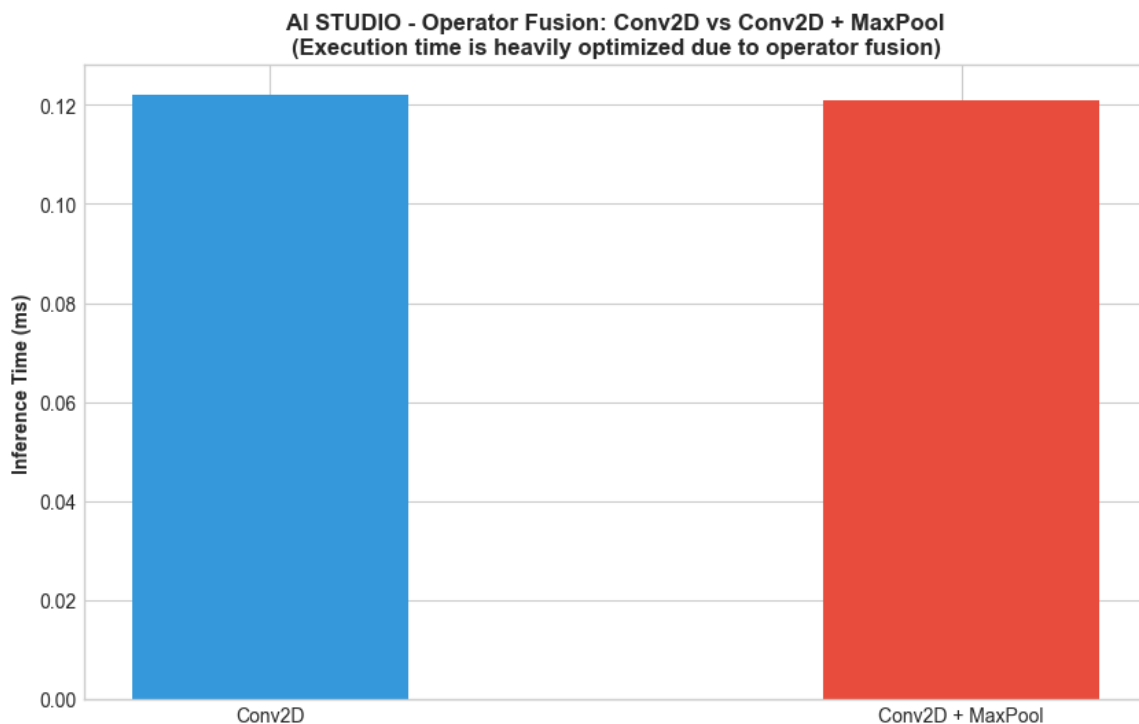
Let's analyze the performance strictly within the AI STUDIO environment.

1.1 What the NPU Accelerates: Operator Fusion

We compare a standard convolution layer with a convolution followed by a Max Pooling layer.

```
In [4]: # Operator fusion (AI Studio)
models_fusion = ['baseline_conv2d_16f_3x3_int8', 'baseline_conv2d_poll']
df_fusion = df_base_aistudio[df_base_aistudio['Model Name'].isin(models_fusion)]
df_fusion['Layer'] = df_fusion['Model Name'].apply(lambda x: 'Conv2D' if 'poll' not in x else 'Conv2D + MaxPool')

fig, ax1 = plt.subplots()
ax1.bar(df_fusion['Layer'], df_fusion['Inference Time (ms)'], color='red')
ax1.set_ylabel('Inference Time (ms)', fontweight='bold')
ax1.set_title('AI STUDIO - Operator Fusion: Conv2D vs Conv2D + MaxPool')
plt.savefig(f'{PLOT_DIR}/1_aistudio_fusion.png', bbox_inches='tight')
plt.show()
display(Markdown("**Why it accelerates:** The NPU is able to fuse the convolution and pooling operations into a single hardware-accelerated block, eliminating the need for a CPU fallback and achieving 0% CPU utilization for these layers.")
```



Why it accelerates: The NPU is able to fuse the MaxPool operator directly into the pipeline of the Conv2D execution. It does not need to write the Conv2D output to RAM and read it back for Pooling, meaning adding pooling costs almost 0 latency overhead.

1.2 What the NPU Accelerates: Convolution Parallelism

Comparing Standard, Pointwise (1x1), and Depthwise convolutions. This highlights how the NPU parallelizes different memory access patterns.

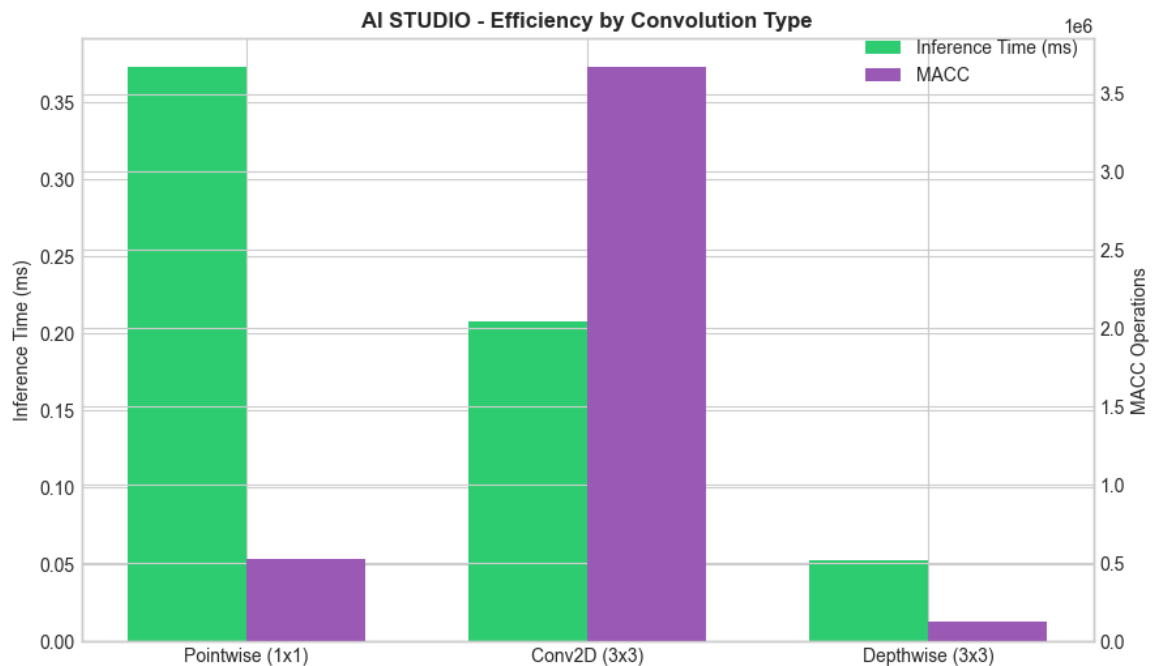
```
In [5]: # Kernel efficiency (AI Studio)
models_type = ['baseline_conv2d_3x3_int8', 'baseline_conv2d_1x1_int8', 'baseline_depthwise_3x3_int8']
df_type = df_base_aistudio[df_base_aistudio['Model Name'].isin(models_type)]
name_map = {
    'baseline_conv2d_3x3_int8': 'Conv2D (3x3)',
    'baseline_conv2d_1x1_int8': 'Pointwise (1x1)',
    'baseline_depthwise_3x3_int8': 'Depthwise (3x3)'
}
df_type['Type'] = df_type['Model Name'].map(name_map)

fig, ax1 = plt.subplots()
ax2 = ax1.twinx()
width = 0.35
x = np.arange(len(df_type['Type']))

ax1.bar(x - width/2, df_type['Inference Time (ms)'], width, label='Inference Time (ms)')
ax2.bar(x + width/2, df_type['MACC'], width, label='MACC', color='#f8766d')

ax1.set_ylabel('Inference Time (ms)')
ax2.set_ylabel('MACC Operations')
ax1.set_xticks(x)
ax1.set_xticklabels(df_type['Type'])
```

```
fig.legend(loc='upper right', bbox_to_anchor=(0.9, 0.9))
plt.title('AI STUDIO - Efficiency by Convolution Type', fontweight=
plt.savefig(f'{PLOT_DIR}/2_aistudio_kernel.png', bbox_inches='tight
plt.show()
```



1.3 Computational Load vs Latency (Compute Bound vs Memory Bound)

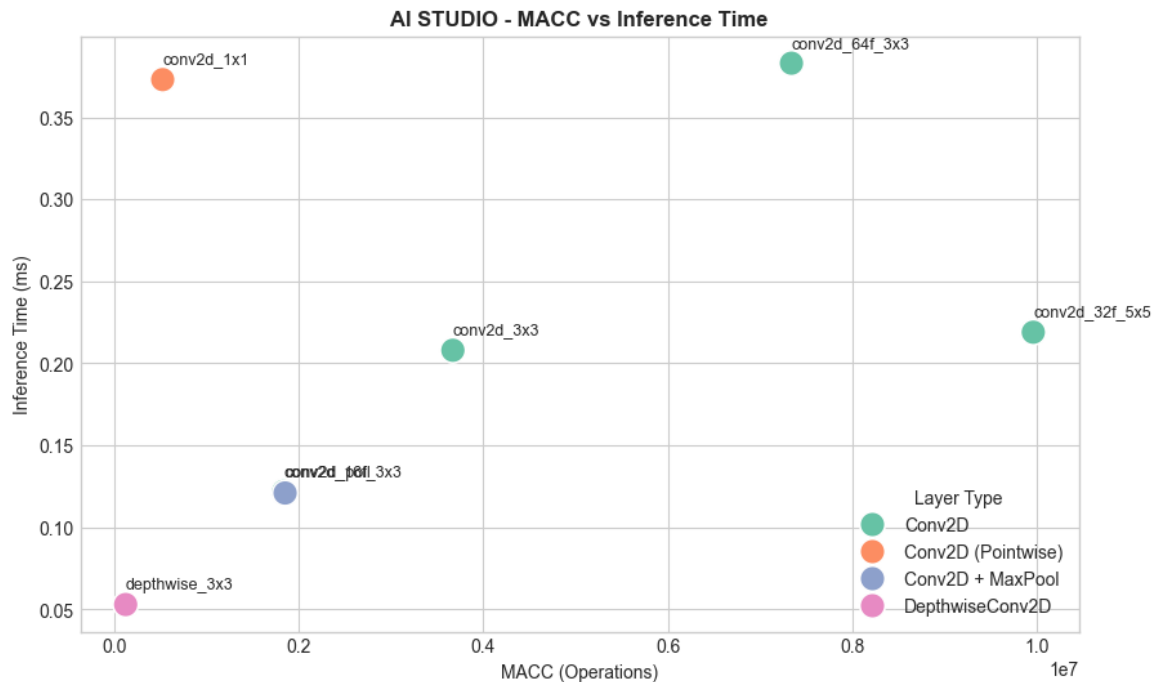
A linear trend represents a compute-bound scenario. Deviations indicate memory-bound latency overheads.

```
In [6]: # Compute Bound vs Memory Bound (AI Studio)
df_valid = df_base_aistudio.dropna(subset=['Inference Time (ms)'])

plt.figure(figsize=(10,6))
sns.scatterplot(data=df_valid, x='MACC', y='Inference Time (ms)', h

for i in range(df_valid.shape[0]):
    plt.text(df_valid['MACC'].iloc[i], df_valid['Inference Time (ms)'.
            df_valid['Model Name'].iloc[i].replace('baseline_', '')
            fontsize=9)

plt.title('AI STUDIO - MACC vs Inference Time', fontweight='bold')
plt.xlabel('MACC (Operations)')
plt.ylabel('Inference Time (ms)')
plt.savefig(f'{PLOT_DIR}/3_aistudio_macc_vs_latency.png', bbox_inch
plt.show()
```



1.4 The Limits of the NPU: RAM Saturation

The NPU is not limitless. Here we evaluate what happens when the model size grows (increasing kernel size). The 7x7 model exceeds the internal ~1.5MB SRAM limit, forcing the compiler to split the data into multiple 'hardware epochs' and leading to an eventual hardware timeout.

```
In [7]: # Architectural Limits (AI Studio)
models_limit = ['baseline_conv2d_3x3_int8', 'baseline_conv2d_32f_5x5']
df_limit = df_base_aistudio[df_base_aistudio['Model Name'].isin(models_limit)]
df_limit['Kernel'] = df_limit['Kernel Size']

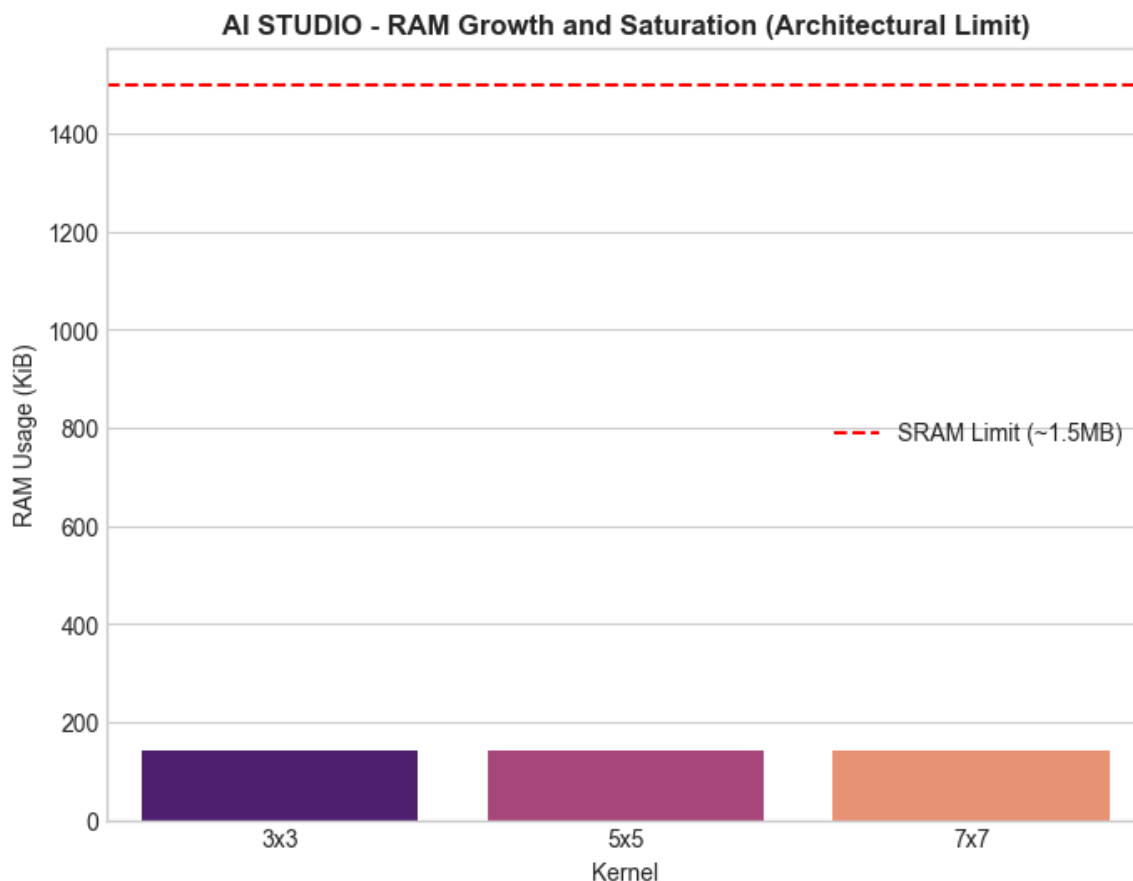
plt.figure(figsize=(8,6))
sns.barplot(data=df_limit.sort_values('RAM Usage (KiB)'), x='Kernel')
plt.axhline(y=1500, color='r', linestyle='--', label='SRAM Limit (~1.5MB)')

plt.title('AI STUDIO - RAM Growth and Saturation (Architectural Limits)')
plt.legend()
plt.savefig(f'{PLOT_DIR}/4_aistudio_ram_saturation.png', bbox_inches='tight')
plt.show()
display(Markdown("**Where the limit lies:** When we hit the 7x7 filter"))
```

```
/var/folders/qn/y4fdcgw97jsbxdyb6s0d79680000gn/T/ipykernel_11193/45514970.py:7: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.
```

```
sns.barplot(data=df_limit.sort_values('RAM Usage (KiB)'), x='Kernel', y='RAM Usage (KiB)', palette='magma')
```

Where the limit lies: When we hit the 7x7 filter, RAM saturation spikes above 1.5MB. The internal SRAM is fully utilized, meaning memory has to be chunked into pieces, causing a significant performance drop or execution failure.

Part 2: Compiler Evolution Comparison

We compare the previous generation compilers (v2.2) against the new generation (v4.0). We separate the analysis into Local Compilers (CubeMX vs AI Studio) and Cloud Compilers (DevCloud AI Core).

```
In [8]: # Compiler Evolution Comparison
df_compare_local = pd.merge(df_base_cubemx[['Model Name', 'Inference Time (ms)']],
                             df_base_aistudio[['Model Name', 'Inference Time (ms)']],
                             on='Model Name', suffixes=('_ (CubeMX v2.2)', '_ (AI Studio v4.0)'))

df_compare_cloud = pd.merge(df_base_aicore2[['Model Name', 'Inference Time (ms)']],
                             df_base_aicore4[['Model Name', 'Inference Time (ms)']],
                             on='Model Name', suffixes=('_ (DevCloud v2.2)', '_ (DevCloud v4.0)'))

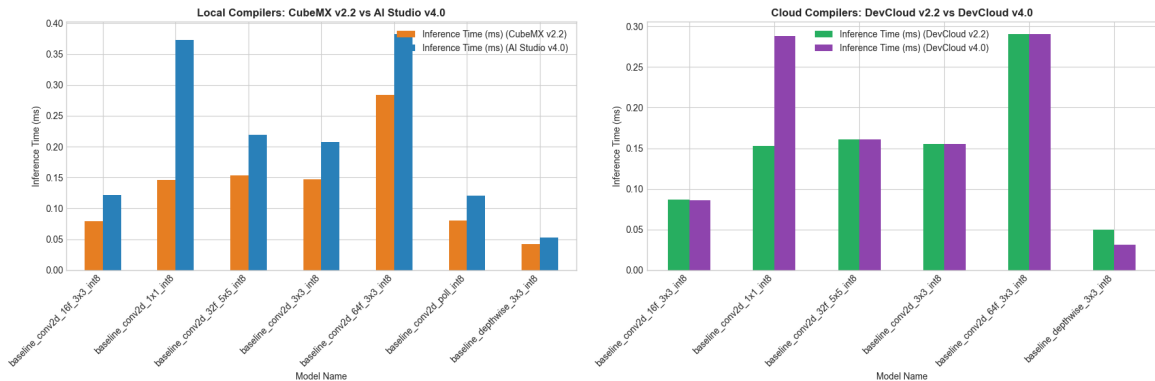
df_compare_local = df_compare_local.dropna().set_index('Model Name')
df_compare_cloud = df_compare_cloud.dropna().set_index('Model Name')

fig, axes = plt.subplots(1, 2, figsize=(18, 6))

df_compare_local.plot(kind='bar', ax=axes[0], color=['#e67e22', '#2980b9'])
axes[0].set_title('Local Compilers: CubeMX v2.2 vs AI Studio v4.0',
                  fontweight='bold', color='red')
axes[0].set_ylabel('Inference Time (ms)', color='red')
axes[0].set_xticklabels(axes[0].get_xticklabels(), rotation=45, ha='right')
```

```
df_compare_cloud.plot(kind='bar', ax=axes[1], color=['#27ae60', '#8e44ad'],
axes[1].set_title('Cloud Compilers: DevCloud v2.2 vs DevCloud v4.0')
axes[1].set_ylabel('Inference Time (ms)')
axes[1].set_xticklabels(axes[1].get_xticklabels(), rotation=45, ha=

plt.tight_layout()
plt.savefig(f'{PLOT_DIR}/5_compiler_comparison.png')
plt.show()
```



Part 3: Estimated vs Actual Real Data

How reliable are the compiler's memory estimates? We compare the AI STUDIO Estimated RAM footprint generated during compilation against the Actual RAM allocated on the board.

```
In [9]: # Estimated vs Real Data (AI STUDIO)
df_est_real = pd.merge(df_base_aistudio[['Model Name', 'RAM Usage (KiB)']],
df_est_aistudio[['Model Name', 'Estimated RAM (KiB)']],
on='Model Name')

# Clean model names for better visualization
df_est_real['Short Name'] = df_est_real['Model Name'].str.replace(' ', '_')
df_est_real.set_index('Short Name', inplace=True)
df_est_real.drop(columns=['Model Name'], inplace=True)
df_est_real.rename(columns={'RAM Usage (KiB)': 'Actual RAM', 'Estimated RAM (KiB)': 'Estimated RAM'})

df_est_real.plot(kind='bar', figsize=(12, 6), color=['#d35400', '#f1c40f'],
plt.title('AI STUDIO: Estimated vs Actual RAM Usage', fontweight='bold')
plt.ylabel('RAM Usage (KiB)')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.savefig(f'{PLOT_DIR}/6_estimated_vs_real_ram.png')
plt.show()
```

